

Matriz de Decisión y Comparación de Modelos

Justificación del gate de dos etapas (score determinístico + modelo de lenguaje) y del modelo elegido para cada etapa, con datos medidos en corridas reales.

1. El gate, en 3 líneas

Input: los campos del lead tal como llegan de Google Sheets (nombre, cargo, empresa, país, tamaño de empresa, email, teléfono, URL de LinkedIn y de la empresa).

Output: `score_rules` (entero de 0 a 100) y `valid` (booleano: ¿tiene el lead un email con formato correcto?).

Regla: si `valid` = false → el lead no pasa a ningún modelo de IA, se guarda como Invalido y termina ahí. Si `valid` = true → `segment_rules` ubica al lead en Caliente (≥ 70), Tibio (40-69) o Frío (< 40), y pasa a la etapa de generación de contenido.

En resumen, antes del detalle

Modelo elegido	GPT-4o-mini
Por qué	Genera textos cortos y estructurados; 16x más barato que GPT-4o y sigue siendo suficiente para esta tarea puntual
Costo por lead	≈ \$0,000165 (menos de 2 centésimas de centavo)
Costo mensual estimado	≈ \$0,26 para 1.000 leads/mes
Ahorro del gate previo	16,7% menos llamadas a la IA, sin gastar nada en leads sin contacto válido

El detalle completo (matriz por etapa, criterios y proyección a mayor escala) está en las secciones siguientes.

2. Arquitectura de dos etapas

Etapas	Motor	Tipo	Costo por caso	Qué decide
1. Compute Rules Score	Code (JavaScript)	Determinístico, sin IA	\$0	Si el lead sigue al pipeline de IA (<code>valid</code>) y a qué segmento pertenece (<code>segment_rules</code>).
2. Basic LLM Chain	gpt-4o-mini	Modelo de lenguaje	≈ \$0.000165	Redacta asunto, cuerpo de email y nota interna de venta.

3. Matriz comparativa por criterio

Criterio	Etapas 1 — Compute Rules Score	Etapas 2 — Basic LLM Chain
Costo por caso	\$0 (regla determinística, no llama ninguna API de IA)	≈ \$0.000165 por lead (gpt-4o-mini, ~700 tokens de entrada / hasta 100 de salida)
% de leads filtrados (medido)	16,7% de los leads no pasan a la Etapa 2 (1 de 6 en la corrida de prueba registrada)	—
Riesgo de falsos negativos	Bajo pero real: durante el desarrollo se detectó y corrigió un bug donde un teléfono válido “tapaba” un email inválido (<code>valid = validEmail validPhone</code>). Se redefinió <code>valid</code> para depender solo de <code>validEmail</code> .	No aplica: esta etapa no descarta leads, solo genera contenido para los que ya pasaron el gate.
Precisión / cómo se validó	No es un modelo probabilístico — es determinístico. Se validó con casos de prueba manuales (emails válidos e inválidos, distintos tamaños de empresa e industrias).	Se valida revisando el JSON de salida. El límite <code>maxTokens=100</code> configurado en el nodo es ajustado para <code>subject+body+nota</code> y puede truncar la respuesta; se recomienda subirlo a ~220.

4. Ahorro medido y estimado

Datos reales de la corrida de prueba: 6 leads procesados por el gate → 1 descartado por invalido (16,7%) → 5 llegaron a la Etapa 2.

Fórmula: $\text{costo_ahorrado} \approx \text{leads_no_agente} \times \text{costo_agente}$

Con el volumen real del proyecto (≈1.000 leads/mes) y la misma proporción de inválidos observada, unos 167 leads/mes evitarían la Etapa 2, ahorrando ≈ \$0,03/mes con gpt-4o-mini.

Lectura honesta del número: el ahorro en dólares es chico porque gpt-4o-mini ya es extremadamente barato. El valor real de este gate no es cuánto ahorra con este modelo puntual, sino que evita gastar en *cualquier* modelo que se use en la Etapa 2: el mismo 16,7% de llamadas evitadas se traduce en montos mucho mayores si en el futuro se usa un modelo más caro.

5. Por qué no hay un segundo gate por calidez (Caliente / Tibio / Frío)

Actualmente todos los leads válidos —sin importar el segmento— pasan por la Etapa 2. Es una decisión de diseño, no un descuido: como gpt-4o-mini ya cuesta fracciones de centavo por lead, un segundo gate que salte la Etapa 2 para los leads Frío ahorraría menos de un centavo cada 100 leads, a cambio de más complejidad en el flujo (un router adicional antes de generar contenido). **Recomendación para escalar:** si el volumen crece uno o dos órdenes de magnitud, o si el modelo de la Etapa 2 se reemplaza por uno premium (GPT-4o o Claude Sonnet) para mejorar la calidad de redacción en Caliente, ahí sí conviene agregar ese segundo gate: Frío recibe una plantilla estática sin IA, y solo Caliente/Tibio consumen el modelo caro.

Precios de referencia (OpenAI/Anthropic) verificados en agosto de 2026; conviene reconfirmarlos antes de presupuestar a mayor escala.

6. Por qué un LLM simple, sin memoria ni tools

La Etapa 2 usa un Basic LLM Chain simple: sin memoria (no guarda historial entre llamadas) y sin tools (no ejecuta acciones ni consulta fuentes externas). No es solo una elección de simplicidad — son dos fuentes de costo distintas, y una de las dos escala mucho peor que la otra.

Configuración	Overhead por llamada	Cómo escala en un lote	Se justifica cuando...
LLM simple (elegido)	Solo el prompt de la tarea (~700 tokens)	Lineal: cada lead cuesta lo mismo, sin importar cuántos se procesaron antes	Cada lead es una tarea independiente y de un solo turno.
AI Agent + Tools (sin memoria)	+ definición de cada tool disponible (~150-300 tokens extra), aunque no se use ninguna	Lineal, pero con un piso más alto por llamada	El modelo necesita ejecutar una acción o consultar algo en tiempo real (hoy, ninguna tool se usa).
AI Agent + Memoria + Tools	Todo lo anterior, más el historial acumulado de llamadas previas en la misma sesión	Cuadrático si la memoria se comparte entre los leads de un mismo lote: cada lead nuevo reenvía el historial completo de los anteriores	Hay una conversación real de ida y vuelta con una persona, que hace falta recordar entre turnos.

Por qué la memoria escala distinto: un modelo de lenguaje no “recuerda” nada por sí solo — cada llamada reenvía todo el historial anterior como parte del input. Para dimensionar el riesgo con números: en un lote de 100 leads, sin memoria el consumo es lineal (≈ 70.000 tokens de entrada, 100 por lead). Si la memoria compartiera sesión entre todos los leads del lote, el lead nº100 tendría que reenviar el historial completo de los 99 anteriores en su propia llamada: el total sube a ≈ 4.030.000 tokens de entrada — 57 veces más, sin ninguna mejora en la calidad de la redacción.

En dólares el monto sigue siendo bajo a este volumen (≈ \$0,61 contra ≈ \$0,02 por lote de 100), pero la curva importa más que el número: al triplicar el volumen del lote (300 leads), el costo sin memoria se triplica (≈ \$0,05), mientras que con memoria compartida se multiplica por nueve (≈ \$5,43) — crece con el cuadrado del volumen, no en proporción directa.

Hay un segundo problema, no solo de costo: si la memoria se comparte entre leads dentro del mismo lote, el contenido generado para un lead queda disponible en el contexto del siguiente — el mismo tipo de mezcla de datos entre leads que ya se detectó y corrigió una vez en este proyecto (ver Seguridad y Resiliencia, referencia sin índice explícito en \$(‘Loop Over Items’).item). Evitar la memoria en esta etapa no es solo más barato: también evita reintroducir ese mismo riesgo.